

Earliest deadline first :

```
#include <stdio.h>

struct Process {
    int id, bt, deadline, period, remaining, next_deadline;
};

void scheduling_edf(struct Process p[], int n) {

    int i, t = 0;

    int max_p = 0;
    for(i = 0; i < n; i++) if(p[i].period > max_p) max_p = p[i].period;

    int hyper_period = 1;
    int found = 0;
    while(!found) {
        found = 1;
        for(i = 0; i < n; i++) {
            if(hyper_period % p[i].period != 0) {
                found = 0;
                break;
            }
        }
        if(!found) hyper_period++;
    }

    int gantt[1000];
    for(i = 0; i < n; i++) {
        p[i].remaining = 0;
        p[i].next_deadline = 0;
    }

    for (t = 0; t < hyper_period; t++) {
        for (i = 0; i < n; i++) {
            if (t % p[i].period == 0) {
                p[i].remaining = p[i].bt;
                p[i].next_deadline = t + p[i].deadline;
            }
        }

        int idx = -1;
        int min_deadline = 9999;
        for (i = 0; i < n; i++) {
            if (p[i].remaining > 0) {
                if (p[i].next_deadline < min_deadline) {
                    min_deadline = p[i].next_deadline;
                    idx = i;
                }
            }
        }
    }
}
```

```

    }
}

if (idx != -1) {
    gantt[t] = p[idx].id;
    p[idx].remaining--;
} else {
    gantt[t] = 0;
}
}

printf("\nGantt Chart (Hyper-period: %d)\n", hyper_period);
for (i = 0; i < hyper_period; i++) printf("-----+");
printf("\n|");
for (i = 0; i < hyper_period; i++) {
    if (gantt[i] == 0) printf("Idle |");
    else printf(" P%-2d |", gantt[i]);
}
printf("\n+");
for (i = 0; i < hyper_period; i++) printf("-----+");
printf("\n0");
for (i = 1; i <= hyper_period; i++) printf("%6d", i);
printf("\n");
}

int main() {
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];
    for (i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Enter P%d [Burst, Deadline, Period]: ", i+1);
        scanf("%d %d %d", &p[i].bt, &p[i].deadline, &p[i].period);
    }

    scheduling_edf(p, n);
    return 0;
}

```

Output:

```

Enter number of processes: 3
Enter P1 [Burst, Deadline, Period]: 3 7 20
Enter P2 [Burst, Deadline, Period]: 2 4 5
Enter P3 [Burst, Deadline, Period]: 2 8 10

Gantt Chart (Hyper-period: 20)
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| P2 | P2 | P1 | P1 | P1 | P3 | P3 | P2 | P2 | Idle | P2 | P2 | P3 | P3 | Idle | P2 | P2 | Idle | Idle | Idle |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
0      1      2      3      4      5      6      7      8      9     10     11     12     13     14     15     16     17     18     19     20

```

Proportional share scheduling:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    int id;
    int burst;
    int shares;
} Process;

void scheduling_proportional_share(Process p[], int n, int total_shares, int
total_burst) {
    srand(time(NULL));
    int time_elapsed = 0;

    printf("\nGantt Chart (Total Time: %d):\n|", total_burst);
    while (time_elapsed < total_burst) {
        int lottery = (rand() % total_shares) + 1;
        int accumulated = 0;

        for (int i = 0; i < n; i++) {
            if (p[i].burst > 0) {
                accumulated += p[i].shares;
                if (lottery <= accumulated) {
                    printf(" P%d |", p[i].id);
                    p[i].burst--;
                    time_elapsed++;
                    break;
                }
            }
        }
    }

    printf("\n0");
    for (int i = 1; i <= time_elapsed; i++) {
        printf("  %2d", i);
    }
    printf("\n");
}

int main() {
    int n, total_shares = 0, total_burst = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    Process p[n];
    for (int i = 0; i < n; i++) {
```

```

        p[i].id = i + 1;
        printf("P%d [Burst Shares]: ", i + 1);
        scanf("%d %d", &p[i].burst, &p[i].shares);
        total_shares += p[i].shares;
        total_burst += p[i].burst;
    }

    scheduling_proportional_share(p, n, total_shares, total_burst);
    return 0;
}

```

Output:

```

Enter number of processes: 3
P1 [Burst Shares]: 6 1
P2 [Burst Shares]: 6 2
P3 [Burst Shares]: 6 3

Gantt Chart (Total Time: 18):
| P2 | P3 | P2 | P3 | P2 | P1 | P2 | P3 | P1 | P3 | P3 | P2 | P2 | P3 | P1 | P1 | P1 | P1 |
0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18

```